Low-Level Design (LLD)

Project: RAG-Based Customer Support Assistant

1. Module-Level Design

The system is decomposed into highly specialized modules to ensure maintainability:

- Document Processing Module: Handles the raw IO stream. It validates MIME types and uses pypdf to extract text while stripping non-UTF8 characters.
- Chunking Module: Implements a sliding window algorithm. It converts text into a list of strings using a chunk_size of 500 tokens and a chunk_overlap of 50 tokens.
- Embedding Module: A wrapper around the SentenceTransformer class. It manages the singleton instance of the all-MiniLM-L6-v2 model to prevent redundant memory loading.
- Vector Storage Module: Interacts with the ChromaDB PersistentClient. It manages collection creation, upserting data, and metadata-based deletion.
- Retrieval Module: Executes the query method on the Chroma collection. It applies an optional metadata filter {"filename": target_file} to restrict the search space.
- Query Processing Module: Formulates the system and user prompts. It prepares the payload for the Groq API call.
- Graph Execution Module: Defined using langgraph.graph.StateGraph. It manages the orchestration of nodes and the persistence of the AgentState.
- HITL Module: A specialized state-handler that monitors the needs_human flag and manages the UI lockout signal.

2. Data Structures

The system relies on these core structures for data integrity:

- **Document Representation:**
    - filename: String (Primary key in logic)
    - content: Bytes/String

- **Chunk Format:**
    - id: UUID (e.g., filename_chunkIndex_randomID)
    - text: String (The raw text segment)
    - metadata: Dictionary { "filename": str, "index": int }
- **Query-Response Schema:**
    - Input: { question: str, session_id: str, target_file: str }
    - Output: { answer: str, sources: list, needs_human: bool }
- **State Object (LangGraph):**

```
question: str
target_file: Optional[str]
intent: str  # 'greeting', 'rag_query', or 'escalate'
answer: str
sources: List[str]
needs_human: bool
```

3. Workflow Design (LangGraph)

The graph follows a **Directed Acyclic Graph (DAG)** structure with conditional branching:

- **Nodes:**
    - router_node: The entry point. Performs intent classification.
    - rag_node: The "Processing Node." Handles vector search and LLM generation.
    - greeting_node: The "Output Node" for low-complexity interactions.
    - hitl_escalation_node: The "Output Node" for high-complexity/frustrated interactions.
- **Edges:**
    - START -> router
    - router -> rag_node (if intent == "rag_query")
    - router -> greeting_node (if intent == "greeting")
    - router -> hitl_escalation_node (if intent == "escalate")
    - Any Node -> END

4. Conditional Routing Logic

The router_node uses Llama 3.1 to evaluate the query against these criteria:

- **Escalation Criteria:**
  - **Low Confidence:** If the query is ambiguous (e.g., "Why is it not working?").
  - **Missing Context:** If the vector DB returns zero results for a technical query.
  - **Explicit Request:** Keywords like "human," "agent," "manager," or "complain."
  - **Sentiment:** Detection of frustration or anger in the query string.
- **RAG Criteria:**
  - Questions containing specific entities or "How to" inquiries related to the ingested documentation.

5. HITL Design

- **Trigger:** Triggered when the hitl_escalation_node is reached.
- **Post-Escalation:** The backend sets needs_human = True. The React frontend receives this flag and replaces the input-bar with an escalation-banner.
- **Integration:** In this prototype, the graph terminates at the escalation node. In a production environment, this node would trigger an external event (webhook) to a human agent's dashboard.

6. API / Interface Design

- **Input Format:** plain text string from the user.
- **Output Format:** plain text string returned to user from the interface.
- **Interaction Flow:** A continuous loop (While True) that accepts input, invokes the LangGraph app with the initial state, and prints the result until the user terminates the session.

7. Error Handling

- **Missing Data:** If target_file is provided but does not exist in ChromaDB, the system defaults to "No relevant information found."
- **No Relevant Chunks Found:** The rag_node checks the length of the retrieved chunks. If 0, it triggers a "soft-escalation" answer: *"I don't have that info; would you like a human?"*
- **LLM Failure:** Wrapped in try-except. If Groq fails (rate limits or API down), the system returns a 500 error with a user-friendly message: "Our AI service is temporarily busy. Please try again."